

Q1) What is deadlock? state & explain the condition for deadlock. explain with example.
explain deadlock prevention, deadlock avoidance, detection, deadlock recovery with example.

• Deadlock :-

- A deadlock is a situation in which two or more processes are permanently blocked because each process is waiting for a resource held by another process.

Ex: Process P1 holds resource R1 & waits for R2
P2 holds resource R2 & waits for R1.
Both will wait forever $\rightarrow$ Deadlock.

• Conditions for deadlock :-

1) Mutual Exclusion :-

- at least one resource must be non-sharable
- Only one process can use the resource at a time.

Eg: A printer can be used by only one process at a time.

2) Hold & wait :-

- A process is holding at least one resource and waits
- waiting for additional resource that is currently held by another process.

Eg: P1 holds scanner & waits for printer
P2 holds printer & waits for scanner

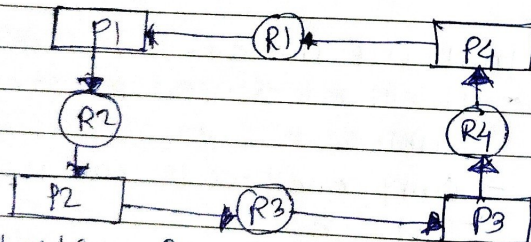
3] No Preemption :-

- A resource cannot be forcibly taken from a process.
- The process must release the resource voluntarily after completing its task.

Eg: a if a process is printing, OS cannot take the printer away forcefully.

4] Circular wait :-

- It is a condition in which a set of processes waiting for a resource held by in such a way that there is a circular chain.
- each process in the chain holding a resource that the next process needs.



P1 is holding R2 & waiting for R1 & soon to prevent deadlock, the OS must eliminate one of four conditions :-

* Deadlock Prevention :-

① Mutual Exclusion :-

- mutual exclusion is not allowed.
- make resources sharable.

Eg: Read-only files

2] Hold and Wait :-

- Avoid hold & wait condition.
- Process must request all resources at once.

Eg: Batch systems

3] Preemption :-

- allow preemption.
- If a process is waiting, its resources are forcibly taken.

Eg: CPU Preemption

4] Circular wait :-

- need to break a circular wait.
- Assign a fixed order to resource type.

Eg: $R1 < R2 < R3$ - Process must request in increasing order.

* Deadlock Avoidance :-

- Deadlock avoidance is a technique in which the operating system checks every request in advance & decides whether granting the request will keep the system in a safe state.

- If the allocation keeps the system safe, the OS grants the resource.
- If the allocation makes the system unsafe, the OS denies the resource allocation to avoid deadlock.

Unsafe state	Safe state
Deadlock	

* Deadlock Detection :-

- Deadlock detection is a technique by which OS allows deadlock to occur but periodically checks whether deadlock has happened.

- It is used in a system where :-

- deadlocks are rare

- allowing a temporary deadlock is acceptable

- methods for deadlock detection :-

1] wait for graph It is used for single instance.

- 2] detection Algorithm is used for multiple instances.

* Deadlock Recovery :-

- Once a deadlock is detected, the system must recover from it.

- To recover it the recovery methods are :-

1] Process Termination :-

- Terminate deadlocked processes.

- or terminate one process at a time

Eg: Kill process P2 $\rightarrow$ R2 released $\rightarrow$ P1 continues.

2] Resource Preemption :-

- Take resources from some processes and give to others.

Eg: OS takes R2 from P2 & gives it to P1.

Q1] What is Semaphore? Justify how Semaphore is used to solve critical section problem.

$\Rightarrow$ • Semaphore :-

- Semaphore is a synchronization tool used in operating systems to manage processes & control access to shared resources.

- a semaphore works using two fundamental operations :-

1) wait () - Decreases semaphore value

2) signal () - Increases semaphore value

* Types of semaphore :-

1. Binary Semaphore (0 or 1) $\rightarrow$ works like a mutex (lock/unlock)

2. Counting Semaphore :- allows multiple processes to access limited instances of a resource.

- Semaphores solve the critical section problem because they satisfy the three requirements :

① mutual Exclusion

- When a process executes wait(S), the semaphore becomes 0

- when another process trying to enter must wait until S becomes 1 again

② provides progress :

- If a critical section is free, semaphore allows a process to enter immediately.

Date _____
Page _____

⑧ Bounded Waiting :-

Processes enter in the order they requested access.

Q3] Write a short note on :-

A] Monitor :-

- A monitor is a high level synchronization construct used in operating systems & concurrent programming that allows safe access to shared data.

- It contains:

- 1) Shared data
- 2) procedure / functions
- 3) initialization code.

- Only one process can execute inside a monitor at a same time, which provides automatic mutual exclusion.

B] Mutex :-

- A mutex is low level synchronization mechanism that is used to provide mutual exclusion.

- Only one thread can lock the mutex at a time.

- If another process thread tried to lock mutex, it must wait until the mutex is unlock.

- Date _____
Page _____
- there are two key operations
 - ① lock() → enters in critical section
 - ② unlock() → exit critical section.

eg:- ensuring one process accesses a printer or a shared variable at a time.

Q4] Explain hardware approach for mutual exclusion with its advantages & disadvantages.

⇒ - The hardware approach provides low-level support for achieving mutual exclusion among processes that share critical section.

- these hardware approaches use special machine instructions provided by CPU.

- these instructions are executed automatically (without interruption).

• Common hardware instructions are:-

① Test & set instructions :-

- this instruction tests & modifies a memory word automatically.

② compare & swap instructions :-

- this instruction compares the content of a memory location with a given value & if they are the same, it swaps it with new value.

• Advantages:-

- ① Simple & fast
- ② works for any no. of processes
- ③ No need for complex software mechanism
- ④ Atomicity guaranteed by hardware

• Disadvantages:-

- ① causes busy waiting
- ② wastes CPU time
- ③ Longer waiting time.

Q5] Write a solution to Reader writer problem using semaphore with readers having priority.

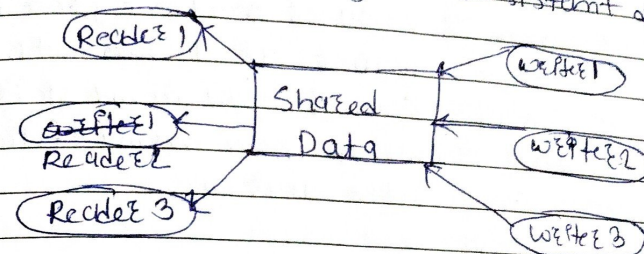
⇒ A In a system, multiple processes want to access a shared data resource.

- Reader:- only read the data
- writers:- can do both read & write data
- multiple readers can read from the database at the same time, but
- when updating a database there can only be one writer
- more than one ~~writers~~ can reader may read shared data, when a writer uses shared data, all other writers & readers must be excluded

* Eg: a library database is accessed by students & librarians

- Students (Readers): can view book details simultaneously

- Librarian: updates book records when the Librarian (Writer) is updating students (Readers) must wait to avoid reading inconsistent data.



Q6] Explain Producer consumer problem & Dining philosopher problem with solution.

⇒ If producer consumer problem is example of classic problems of synchronization.

- Producer process produce a data item that consumer consumes later.
- A consumer process removes item from the buffer & uses them.
- Both processes share the same buffer, so proper synchronization is required.
- buffer is used between producer & consumer.
- producer-consumer is also called as bounded buffer problem.

* Producer-consumer problem

- Producer should not add items when buffer is full.
- Consumer should not remove item when buffer is empty.

- Both must access buffer mutually & exclusively.

* To solve this problem three semaphores are used:

- mutex - ensures only one process uses a buffer at a same time.
- Empty - counts how many slots are available
- Full - counts how many filled slots are available

2] Dining philosopher problem is one of classical process synchronization problem.

- Five philosophers sit around a table with one fork between each pair.
- If all philosophers pick the same fork first, then none get the second fork $\rightarrow$ deadlock occurs.

• Solution:-

here each fork is treated as a semaphore with value 1.

A philosopher must "wait" (take permission) before picking a fork, & "signal" (release permission) after eating.

Q7] What is need of process synchronization explain semaphore in detail.

$\Rightarrow$ Need of process synchronization:-

- ① To achieve mutual exclusion
 - allows only one process to access a critical section at a time.
- ② To avoid deadlock & starvation:-
 - proper synchronization techniques help in preventing process blocking
- ③ To ensure reliable communication:-
 - required for correct transfer of data & cooperation b/w processes.
- ④ To coordinate process execution
 - helps processes - execute in a proper sequence.
- ⑤ To maintain data consistency.
 - ensures shared data or files remains consistent even when accessed by many processes simultaneously.

Q8] Write & explain deadlock avoidance Banker's algorithm.

- $\Rightarrow$
- Banker's algorithm is the deadlock avoidance algorithm.
 - It is named as a banker because it works like a bank:-
 - a bank gives a loan only if it can stay safe & can satisfy all customers in some order.

Simplarily,

- OS allocates resources only if after allocation the system remains safe.
- Deadlock avoidance ensures that the system never enters in unsafe state.
- Components used in Banker's algorithm:-

- 1) Available:- Total no. of free resources.
 - 2) Max:- maximum resources each process needs.
 - 3) Allocation:- Resources currently allocated to each process.
 - 4) Need:- Remaining resources a process requires.
- OS uses Banker's algorithm to allocate resource only if the system remains safe after allocation.